

ANHANGUERA – CAMPUS GUAJAJARAS
THIAGO CONEGUNDES MORAIS - 3510640001
SISTEMA DE INFORMAÇÃO

PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA:
PROGRAMAÇÃO WEB II

BELO HORIZONTE - MINAS GERAIS
MARÇO/2026

THIAGO CONEGUNDES MORAIS - 3510640001
PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA
PROGRAMAÇÃO WEB II

Trabalho acadêmico apresentado ao Curso de Bacharelado em Sistema de Informação da Faculdade Anhanguera como requisito para obtenção de pontuação semestral.

Mediador Pedagógico: Rafael R. Fassula

BELO HORIZONTE - MINAS GERAIS
MARÇO/2026

SUMÁRIO

INTRODUÇÃO	1
OBJETIVO	1
DESENVOLVIMENTO.....	2
CONFIGURAÇÕES INICIAIS DAS VERSÕES.....	2
VARIÁVEIS DE SISTEMA	3
CONFIGURAÇÃO DO WILD FLY NO ECLIPSE	3
CRIAÇÃO DO PROJETO FACULDADE	7
ESTRUTURA DE PASTAS E ARQUIVOS DO PROJETO FACULDADE	9
RESULTADO FINAL PARCIAL	14
RESULTATO FINAL OFICIAL APÓS EXECUÇÃO.....	14
CONSIDERAÇÕES FINAIS.....	15
REFERÊNCIAS.....	16

INTRODUÇÃO

Segundo Helder Guimarães Aragão (2012), Java é uma linguagem de programação utilizada como ferramenta para desenvolver aplicações. Em certos casos são utilizados frameworks. Framework é um conjunto estruturado de pastas, ferramentas, bibliotecas e componentes reutilizáveis projetados para acelerar o desenvolvimento e garantir padrões de código.

Durante o desenvolvimento do trabalho foi utilizado o Framework **JSF** (Java Server Faces); no qual; pode ser usado na construção de interfaces gráficas visando à interação do usuário com a aplicação. (DEV.MEDIA)

Para a construção do projeto foi necessário à escolha de uma IDE (Integrated Development Enviroment / Ambiente de Desenvolvimento Integrado); nesse sentido foi utilizado o Eclipse para escrita dos códigos e execução de testes.

Para simular um ambiente de servidor web na máquina local foi utilizado o WildFlay, sua função é executar e apresentar as aplicações web construídas na IDE Eclipse utilizando a linguagem Java.

OBJETIVO

1- Construir uma aplicação Web utilizando as seguintes tecnologias: Linguagem de programação Java; FrameWork JSF; IDE Eclipse e Servidor WildFlay.

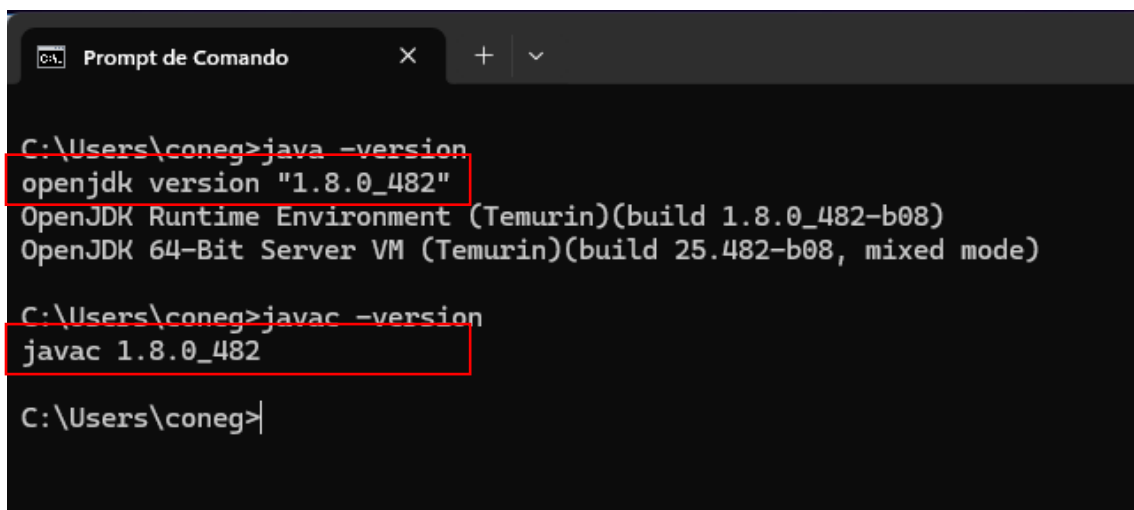
2- Elaborar uma página web para cadastro de faculdade utilizando os arquivos: Faculdade.xhtml representando a parte visual – Front End; Faculdade.java representando a parte de Back End; FaculdadeMB.java para fazer a interconexão entre os arquivos Front End e Back End; Estilo.css para trazer cor, formatação e estilo específico.

DESENVOLVIMENTO

Ao dar início ao desenvolvimento do trabalho, foi necessário seguir criteriosamente as configurações de montagem do ambiente Java, bem como as variáveis do sistema, em conjunto com as configurações do eclipse e servidor Wildfly.

CONFIGURAÇÕES INICIAIS DAS VERSÕES:

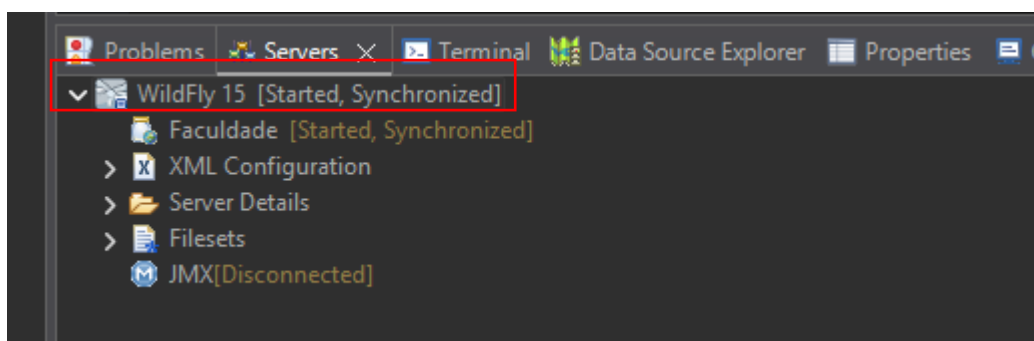
- **JDK 1.8.0.482** - confirmado pelo comando: **javac -version**
- **JRE 1.8.0.482** - confirmado pelo comando: **java -version**
- **WildFly 15.**
- **Eclipse IDE 2025;**



```
C:\Users\coneg>java -version
openjdk version "1.8.0_482"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_482-b08)
OpenJDK 64-Bit Server VM (Temurin)(build 25.482-b08, mixed mode)

C:\Users\coneg>javac -version
javac 1.8.0_482

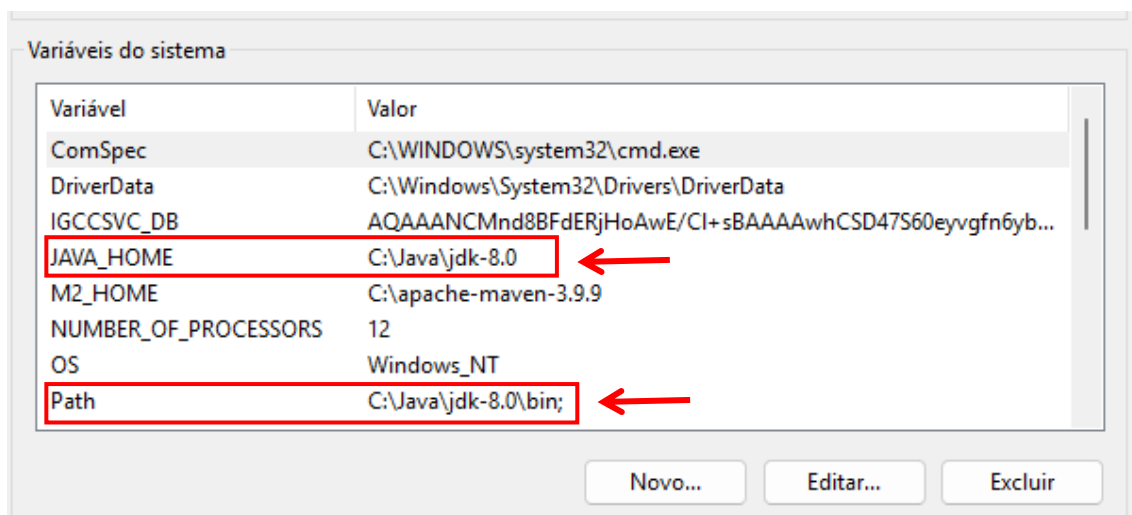
C:\Users\coneg>
```



OBSERVAÇÃO

É importante ressaltar que, o Java 1.8... é compatível com o WildFly 15 devido a isso, não foi utilizado o WildFly da versão 10 como sugerido no documento orientador.

VARIÁVEIS DE SISTEMA



Para realizar essa manobra é importante saber que, as variáveis de sistema existem para configurar o ambiente de execução do sistema operacional Windows e dos programas envolvidos, nesse caso usamos o JDK (Java Development Kit) 12.

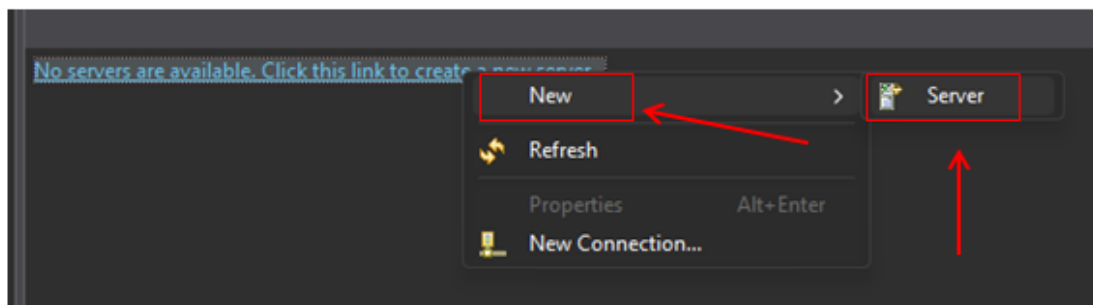
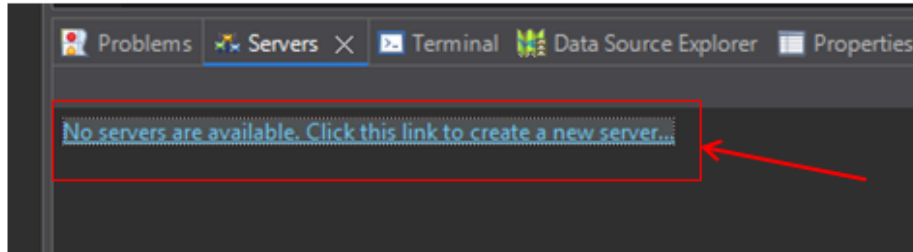
Cada variável tem uma função específica. O **JAVA_HOME** indica onde o JDK está instalado. Já o **Path** lista de diretórios onde o sistema procura pelos arquivos executáveis.

As variáveis **JAVA_HOME** e **Path** foram definidas manualmente. Sem a instalação correta do JDK e a configuração da variável de sistema, não é possível executar o Wildfly.

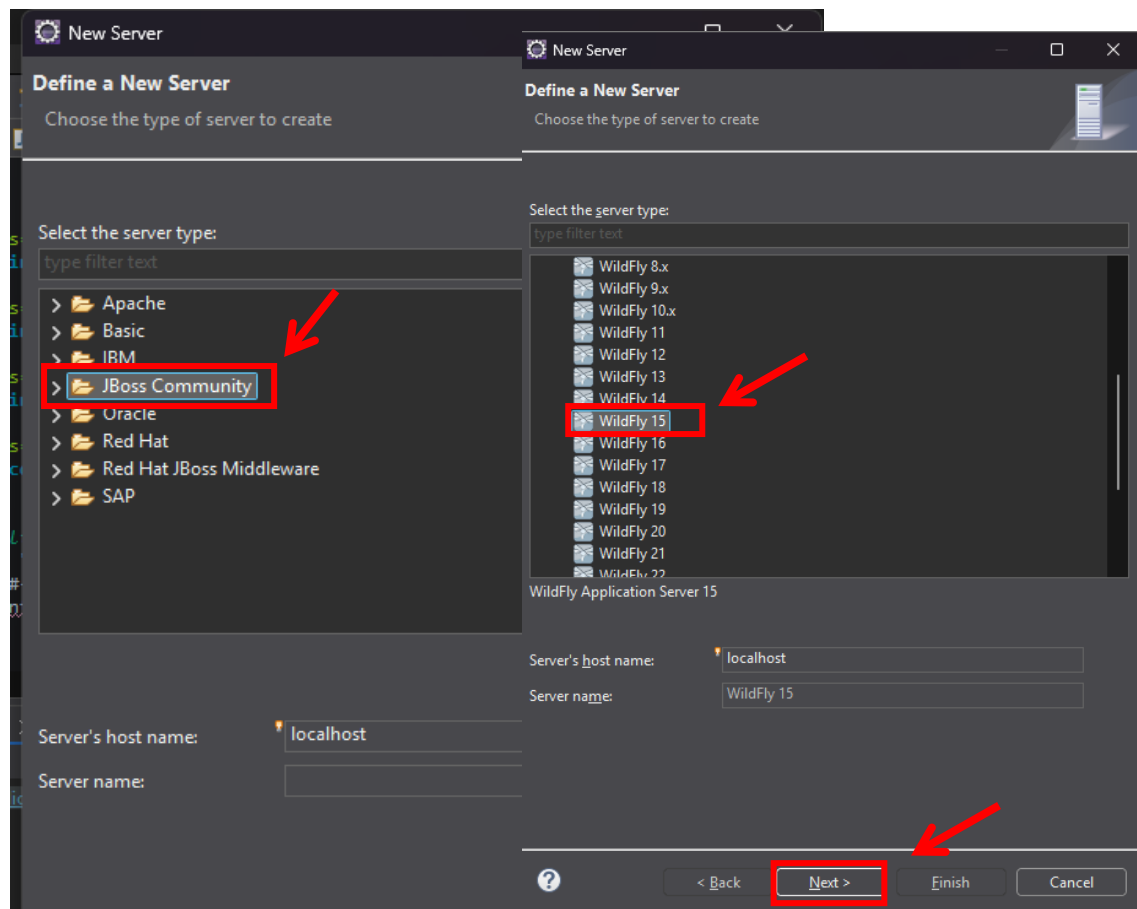
CONFIGURAÇÃO DO WILD FLY NO ECLIPSE

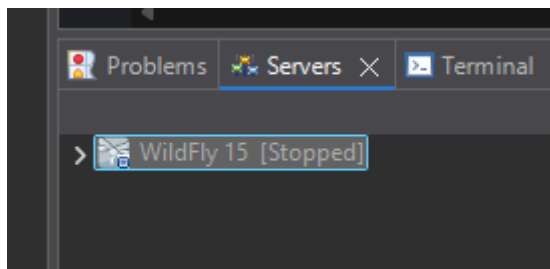
Para utilizar o servidor WildFly é necessário fazer o download com o arquivo executável. Esse servidor deve ser configurado na IDE Eclipse.

No Eclipse, na parte inferior, na opção server, clique na escrito azul: ***“Click this link to create a new server”*** assim como indicado abaixo.



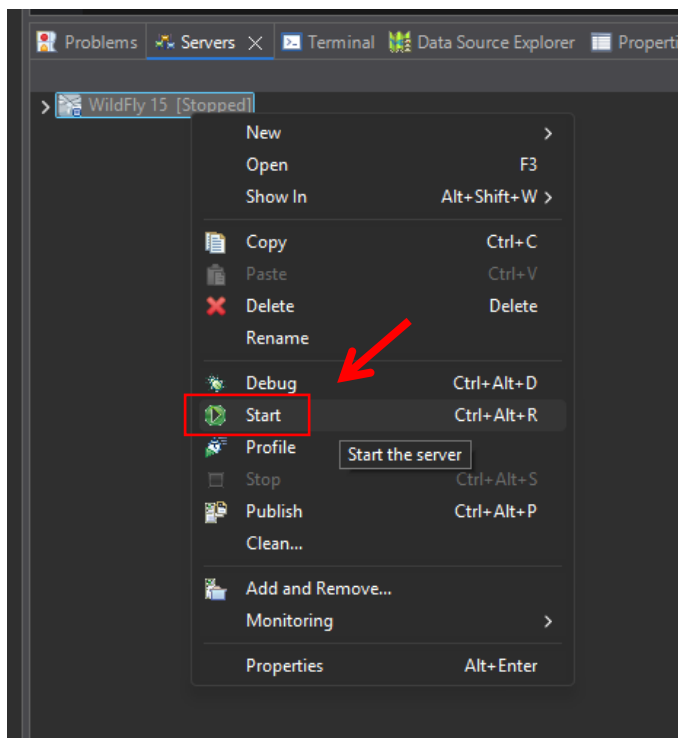
Na opção JBoss Community busque por WildFly15.





EXECUÇÃO DE TESTE

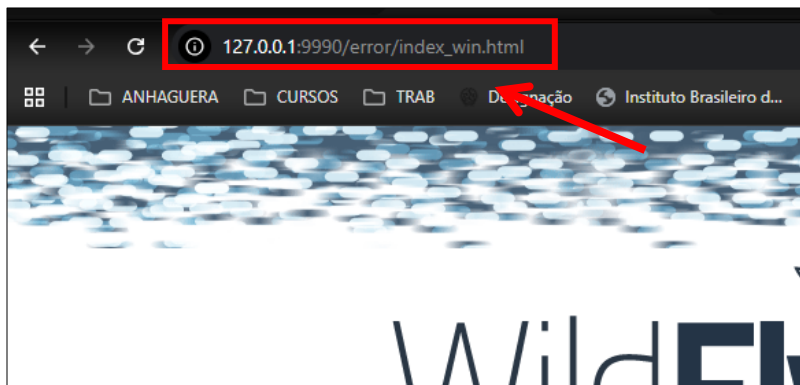
Ao clicar com o botão direito na opção Start temos o seguinte resultado



Em seguida será iniciado o processo de execução.

```
19:33:32,146 INFO [org.jboss.as.clustering.infinispan] (ServerService Thread Pool -- 75) WFLYCLINF0002: Started client-mappings cache from ... containe
19:33:33,564 INFO [org.wildfly.extension.undertow] (ServerService Thread Pool -- 76) WFLYUT0021: Registered web context: '/Faculdade' for server 'default'
19:33:33,576 INFO [org.jboss.as.server] (ServerService Thread Pool -- 43) WFLYSRV0010: Deployed "Faculdade.war" (runtime-name : "Faculdade.war")
19:33:33,662 INFO [org.jboss.as.server] (Controller Boot Thread) WFLYSRV0212: Resuming server
19:33:33,668 INFO [org.jboss.as] (Controller Boot Thread) WFLYSRV0060: Http management interface listening on http://127.0.0.1:9990/management
19:33:33,668 INFO [org.jboss.as] (Controller Boot Thread) WFLYSRV0051: Admin console listening on http://127.0.0.1:9990
19:33:33,668 INFO [org.jboss.as] (Controller Boot Thread) WFLYSRV0025: WildFly Full 15.0.0.Final (WildFly Core 7.0.0.Final) started in 29524ms - Starte
```

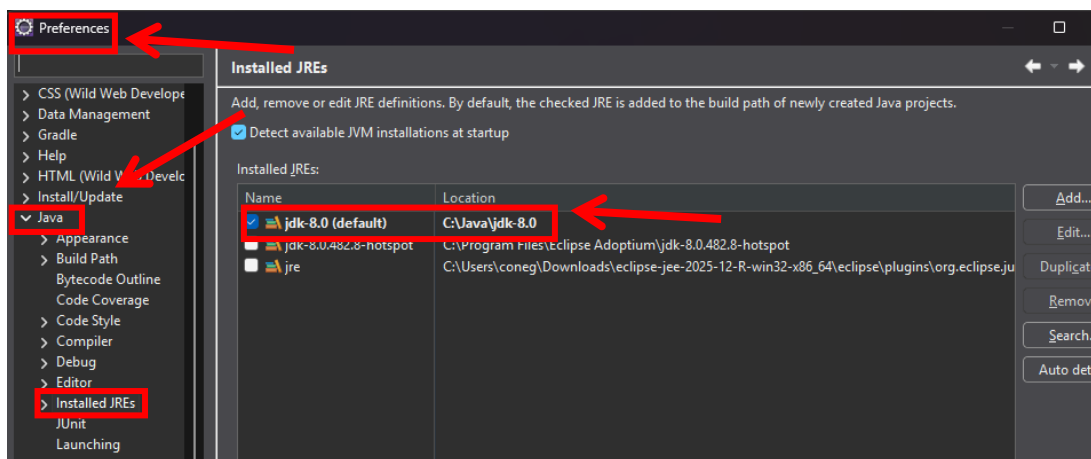
Ao analisar a porta: <http://127.0.0.1:9990> temos o seguinte resultado:



Ao digitar na barra de endereço: "localhost:8080" temos o mesmo resultado.



Na plataforma Eclipse, ao fazer o caminho: **Window, Preferences, Java, Instals JREs**, deve ser confirmado que estamos usando o jdk 8.0 assim como descrito na imagem abaixo:



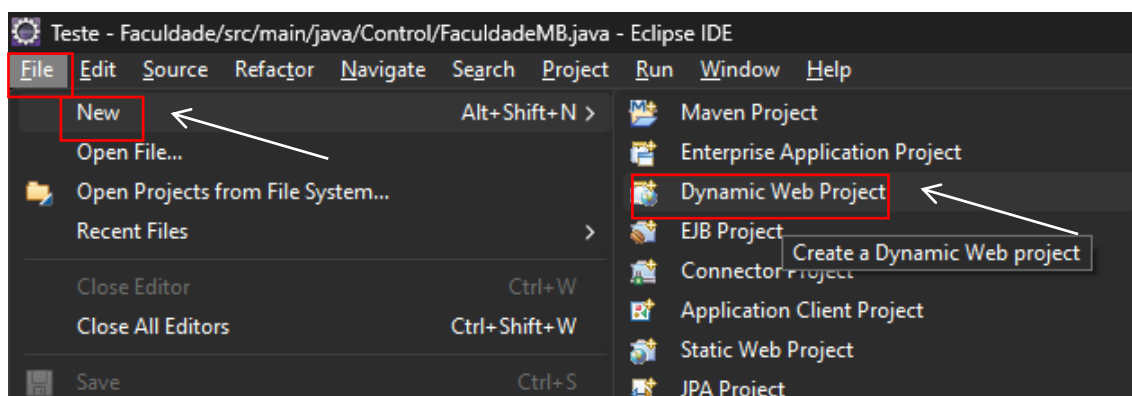
OBSERVAÇÃO:

Ao falar: “Java 8”, temos a versão 1.8.0..., sendo a versão apresentada no prompt de comando – cmd descrita na imagem da página 2 desse relatório. Essa versão é a mesma apresentada no eclipse, porém com nomenclatura diferente.

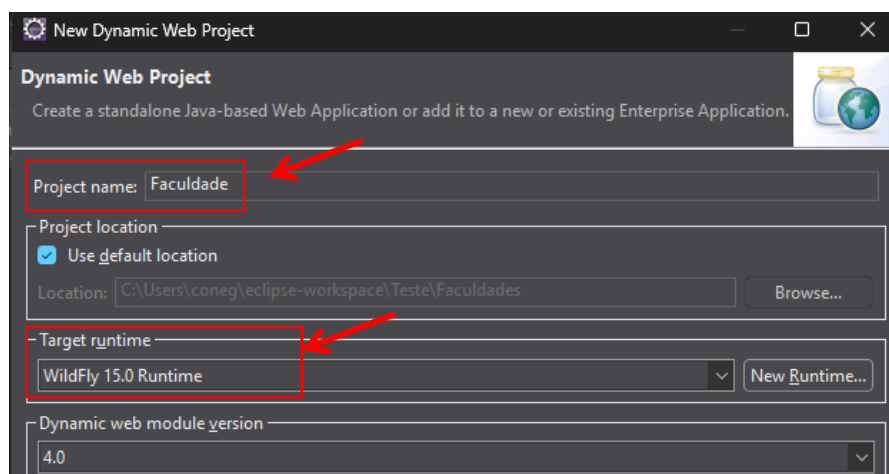
Nome	Versão interna <i>Versão apresentada no cmd</i>	Significado <i>Versão apresentada no eclipse</i>
JDK 8	1.8.0_482	Java Development Kit versão 8 (jdk 8)

CRIAÇÃO DO PROJETO FACULDADE

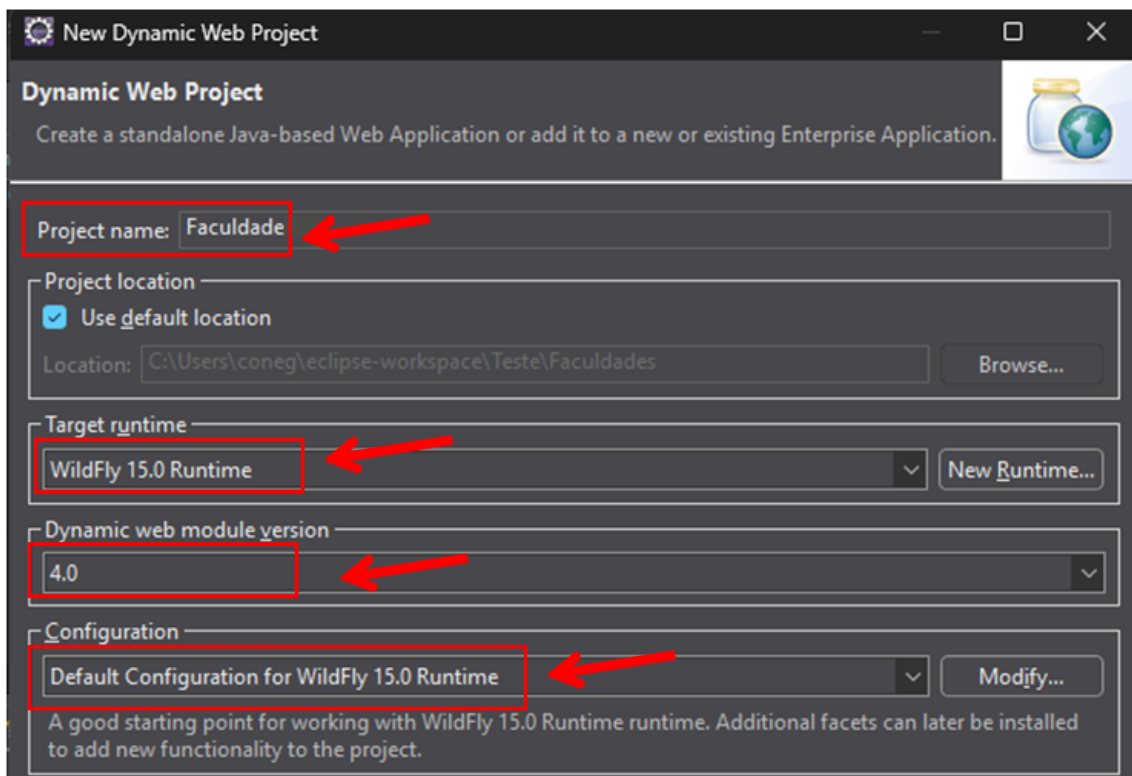
Após a criação do servidor Wildfly, será necessário criar o projeto Faculdade, para isso clique em **File**, **New** e **Dynamic Web Project**, assim como demonstrado abaixo:



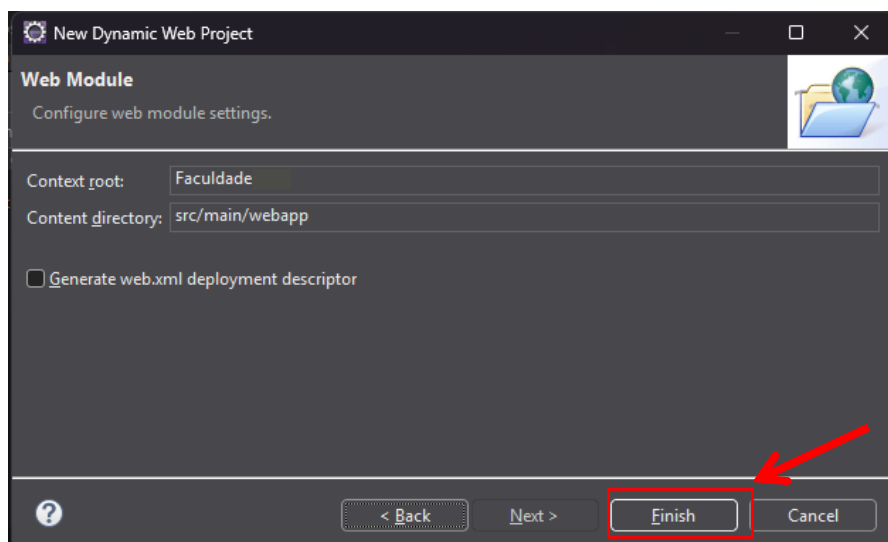
Em seguida, na próxima tela será inserido o nome do projeto Faculdade. Observe que todas as opções correlacionadas a configuração do WildFly irão aparecer no momento da criação do projeto.



Apresentação completa da imagem:



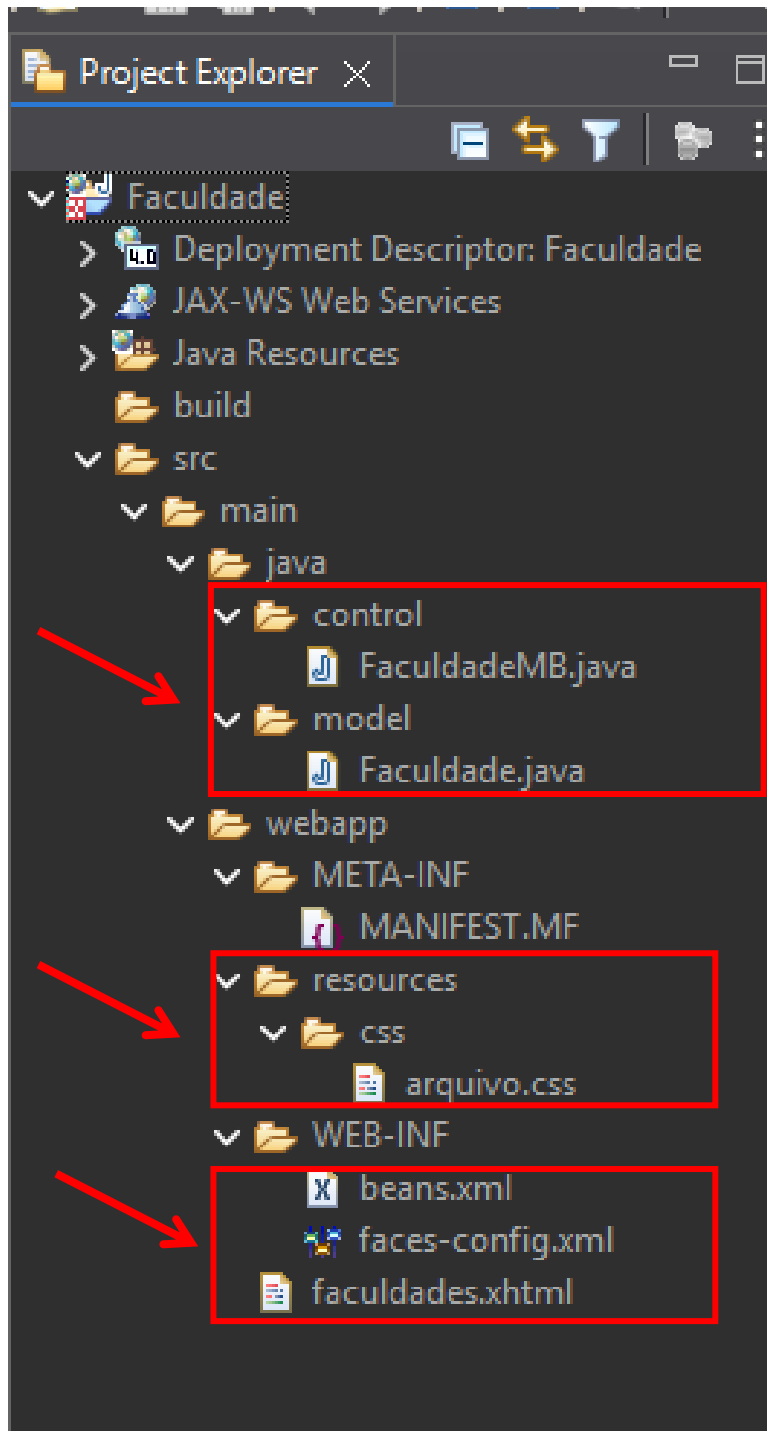
Em seguida clique em finalizar.



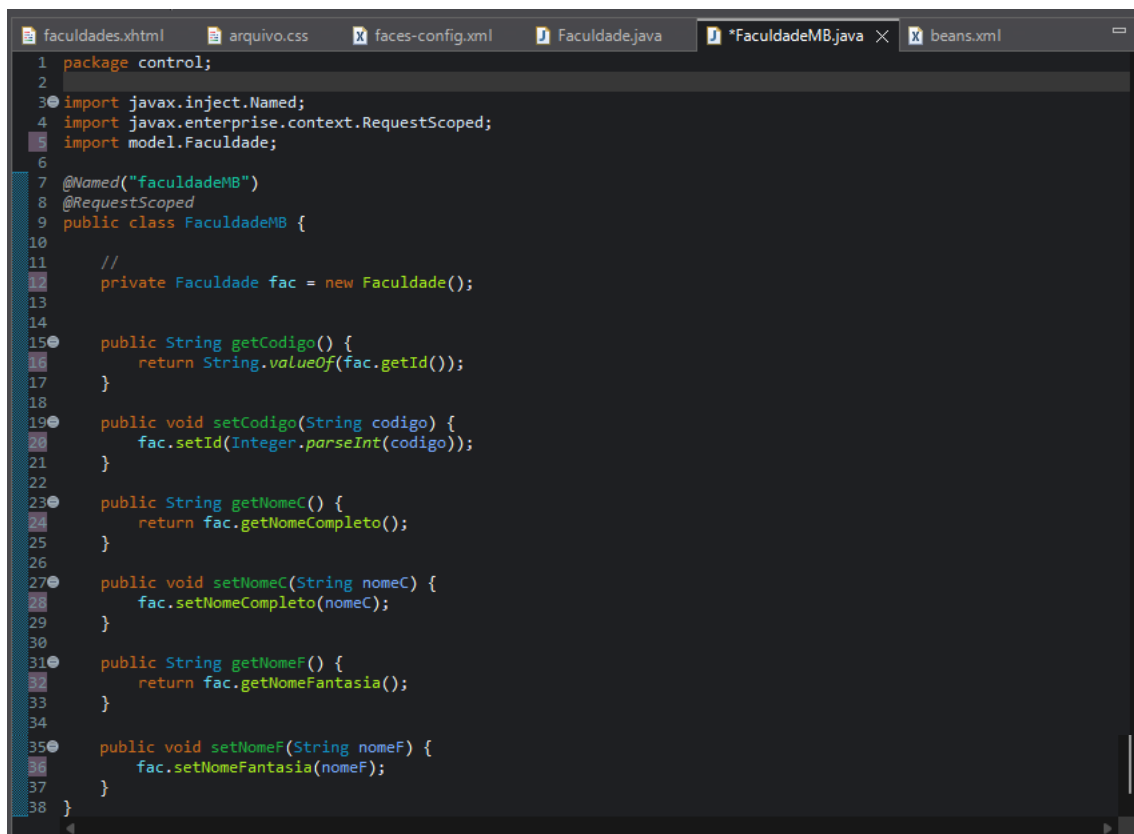
Essa é a forma correta de criar o projeto e configurar o servidor Wild Flay.

ESTRUTURA DE PASTAS E ARQUIVOS DO PROJETO FACULADE

Para que o projeto funcione deve ser seguida essa estrutura de pastas com essa organização de arquivos.

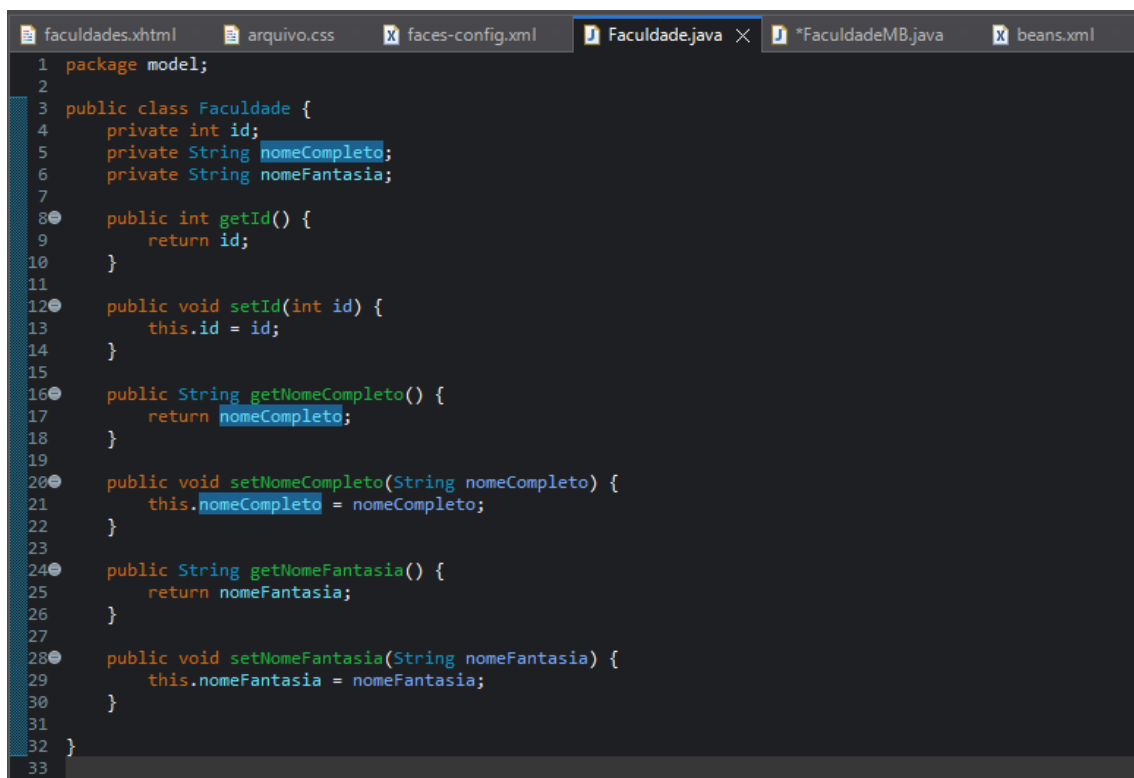


APRESENTAÇÃO DO ARQUIVO FACULDADE_MB.JAVA:



```
1 package control;
2
3 import javax.inject.Named;
4 import javax.enterprise.context.RequestScoped;
5 import model.Faculdade;
6
7 @Named("faculdadeMB")
8 @RequestScoped
9 public class FaculdadeMB {
10
11     //
12     private Faculdade fac = new Faculdade();
13
14
15     public String getCodigo() {
16         return String.valueOf(fac.getId());
17     }
18
19     public void setCodigo(String codigo) {
20         fac.setId(Integer.parseInt(codigo));
21     }
22
23     public String getNomeC() {
24         return fac.getNomeCompleto();
25     }
26
27     public void setNomeC(String nomeC) {
28         fac.setNomeCompleto(nomeC);
29     }
30
31     public String getNomeF() {
32         return fac.getNomeFantasia();
33     }
34
35     public void setNomeF(String nomeF) {
36         fac.setNomeFantasia(nomeF);
37     }
38 }
```

APRESENTAÇÃO DO ARQUIVO FACULDADE.JAVA



```
1 package model;
2
3 public class Faculdade {
4     private int id;
5     private String nomeCompleto;
6     private String nomeFantasia;
7
8     public int getId() {
9         return id;
10    }
11
12    public void setId(int id) {
13        this.id = id;
14    }
15
16    public String getNomeCompleto() {
17        return nomeCompleto;
18    }
19
20    public void setNomeCompleto(String nomeCompleto) {
21        this.nomeCompleto = nomeCompleto;
22    }
23
24    public String getNomeFantasia() {
25        return nomeFantasia;
26    }
27
28    public void setNomeFantasia(String nomeFantasia) {
29        this.nomeFantasia = nomeFantasia;
30    }
31
32 }
33
```

APRESENTAÇÃO DO ARQUIVO FACULDADES.XHTML

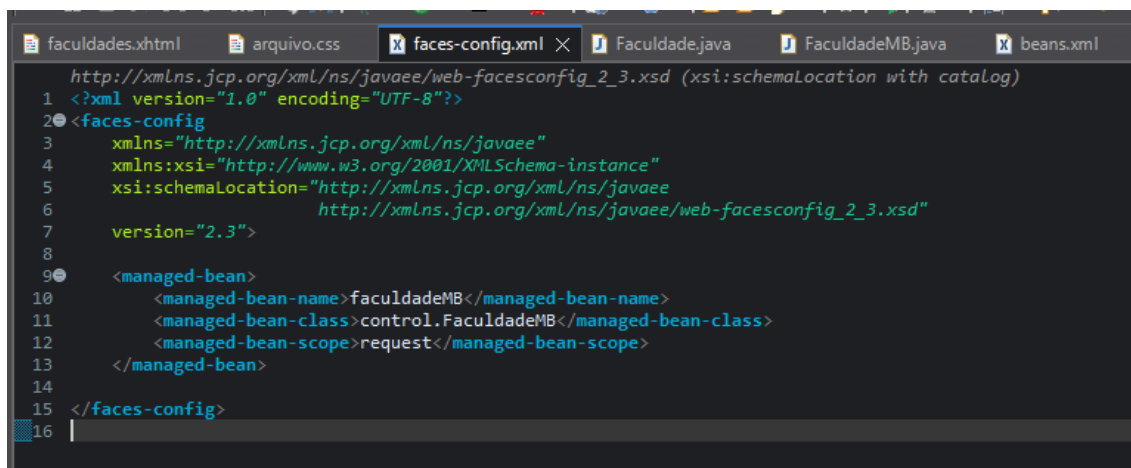
```
faculdades.xhtml | arquivo.css | faces-config.xml | Faculdade.java | FaculdadeMB.java
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml" xmlns:h="http://xmlns.jcp.org/jsf/html">
3 <h:head>
4   <meta charset="UTF-8"/>
5   <title>Cadastro de Faculdade</title>
6   <h:outputStylesheet library="css" name="arquivo.css" />
7 </h:head>
8 <h:body>
9   <h:form>
10    <p class="rotulo">Código:
11      <h:inputText value="#{faculdadeMB.codigo}" class="entrada" />
12    </p>
13    <p class="rotulo">Nome completo:
14      <h:inputText value="#{faculdadeMB.nomeC}" class="entrada" />
15    </p>
16    <p class="rotulo">Nome fantasia:
17      <h:inputText value="#{faculdadeMB.nomeF}" class="entrada" />
18    </p>
19    <p class="botao">
20      <h:commandButton value="Submit" action="faculdades" />
21    </p>
22  </h:form>
23  <p id="resultado">Faculdade cadastrada: <br/>
24    Código: "#{faculdadeMB.codigo}"<br/>
25    Nome: "#{faculdadeMB.nomeC}"<br/>
26    Nome fantasia: "#{faculdadeMB.nomeF}"<br/>
27  </p>
28 </h:body>
29 </html>
```

APRESENTAÇÃO DO ARQUIVO.CSS PARA ESTILO DE PÁGINA

```
faculdades.xhtml | arquivo.css | faces-co
1 @charset "UTF-8";
2 .rotulo {
3   font-family: Arial, Sans-serif;
4   text-transform: uppercase;
5   font-size: 12pt;
6   font-weight: bold;
7   color: red;
8 }
9
10 .entrada {
11   border: 1px inset;
12   background-color: #D3D3D3;
13   color: #000000;
14   display: block;
15 }
```

Foi alterado a cor da fonte para vermelho visando dar destaque, comprovando que o estilo de fato foi inserido e está executando corretamente.

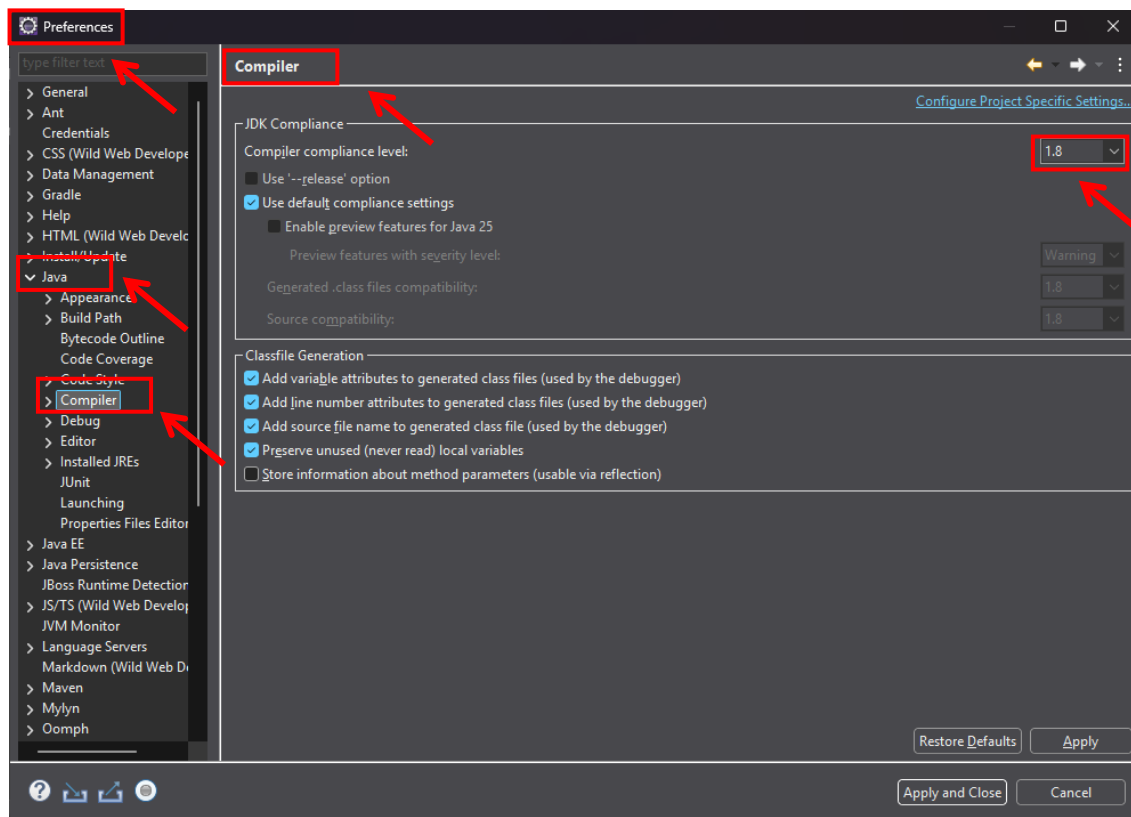
APRESENTAÇÃO DO ARQUIVO FACES-CONFIG.XML



OBSERVAÇÕES

O arquivo faces-config.xml é o arquivo de configuração principal do JSF (Java Server Faces) ele organiza e configura informando como o JSF deve se comportar. Ele informa ao JSF que existe um arquivo faculdaeMB.java e faz a requisição deles.

Para a execução do projeto deve ser verificado o Java Compiler. Ao acessar o caminho Window, Preferences, Java, Compiler, temos que garantir que o compilador esteja na versão 1.8, assim como descrito na imagem abaixo.



EXPLICAÇÃO BREVE DO CÓDIGO

No arquivo faculdade.xhtml temos um formulário que contém um input de texto apresentado pela tag: `<h:inputText value="#{faculdadeMB.nomeC}"`, esse input recebe o nome completo da faculdade digitado pelo usuário.

```
<p class="rotulo">Nome completo:
  <h:inputText value="#{faculdadeMB.nomeC}" class="entrada" />
</p>
```

O conteúdo armazenado em **"nomeC"** é levado para o arquivo: Faculdade MB.java.

Ao chegar no arquivo faculdadeMB, o método `setNomeC` é acionado, nele temos um objeto `"fac"`, sendo do tipo `faculdade`. Ele aciona o método `"setNomeCompleto"` armazenando na variável o conteúdo digitado pelo usuário.

```
public void setNomeC(String nomeC) {
    fac.setNomeCompleto(nomeC);
}
```

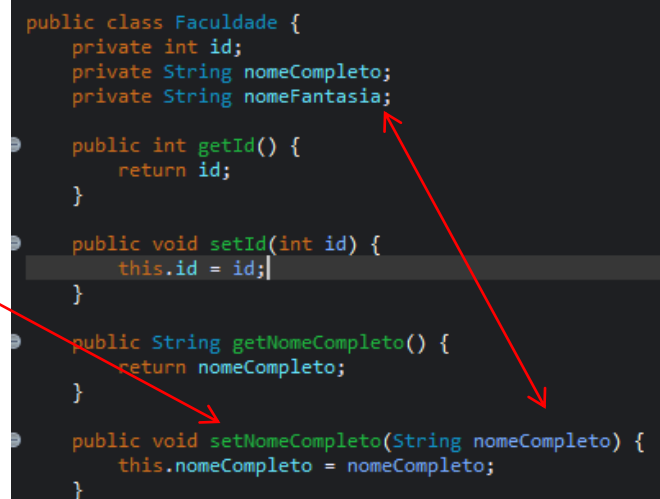
```
public class Faculdade {
    private int id;
    private String nomeCompleto;
    private String nomeFantasia;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNomeCompleto() {
        return nomeCompleto;
    }

    public void setNomeCompleto(String nomeCompleto) {
        this.nomeCompleto = nomeCompleto;
    }
}
```



Para obter esses dados são utilizados os métodos: `"getNomeCompleto"` para o arquivo `faculdade`; já o arquivo `faculadeMB` é utilizado o `getNomeC`.

```
public String getNomeC() {
    return fac.getNomeCompleto();
}
```

```
public String getNomeCompleto() {
    return nomeCompleto;
}
```



Isso acontece para todos os dados informados pelo usuário na tela de cadastro.

RESULTADO FINAL PARCIAL



A screenshot of a web browser window showing a registration form. The browser's address bar displays 'localhost:8080/Faculdade/faculdades.xhtml'. The browser's tab bar shows several tabs: 'ANHAGUERA', 'CURSOS', 'TRAB', 'Designação', and 'Insti'. The form contains three input fields with red labels: 'CÓDIGO:' with the value '0', 'NOME COMPLETO:' which is empty, and 'NOME FANTASIA:' which is empty. Below these fields is a 'Submit' button. At the bottom of the form, the text 'Faculdade cadastrada:' is followed by 'Código: "0"', 'Nome: ""', and 'Nome fantasia: ""'.

CÓDIGO:
0

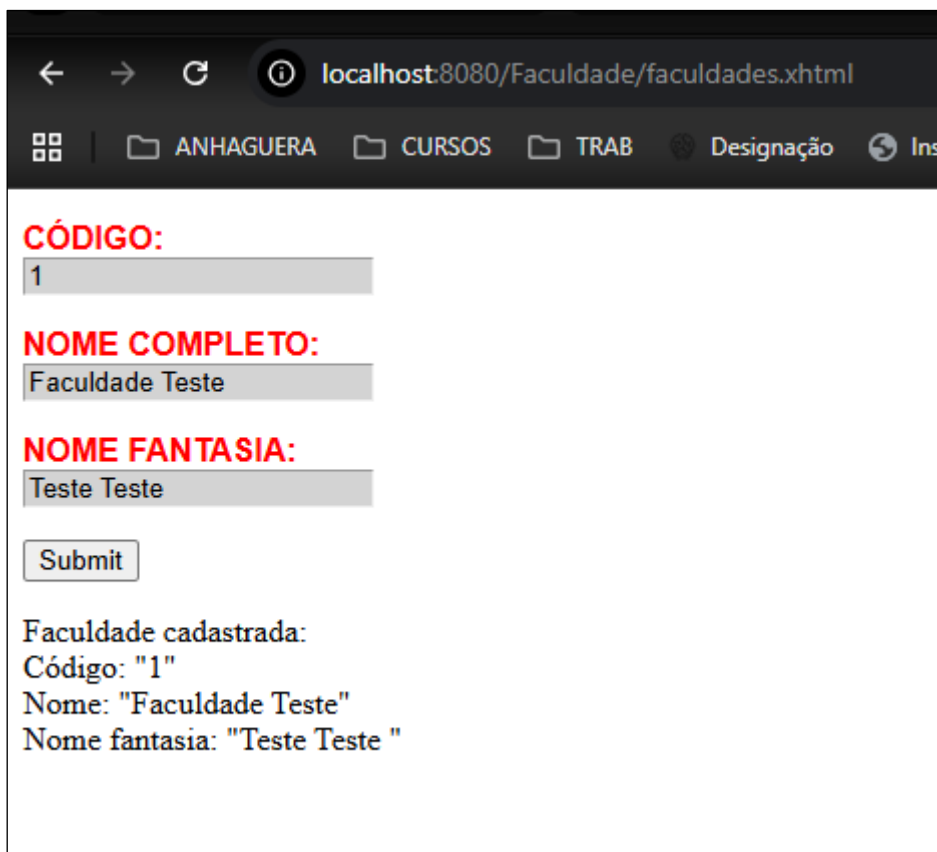
NOME COMPLETO:

NOME FANTASIA:

Submit

Faculdade cadastrada:
Código: "0"
Nome: ""
Nome fantasia: ""

RESULTATO FINAL OFICIAL APÓS EXECUÇÃO



A screenshot of a web browser window showing the same registration form as above, but with updated data. The browser's address bar and tab bar are identical. The 'CÓDIGO:' field now contains the value '1'. The 'NOME COMPLETO:' field contains the text 'Faculdade Teste'. The 'NOME FANTASIA:' field contains the text 'Teste Teste'. The 'Submit' button remains. At the bottom, the text 'Faculdade cadastrada:' is followed by 'Código: "1"', 'Nome: "Faculdade Teste"', and 'Nome fantasia: "Teste Teste "'.

CÓDIGO:
1

NOME COMPLETO:
Faculdade Teste

NOME FANTASIA:
Teste Teste

Submit

Faculdade cadastrada:
Código: "1"
Nome: "Faculdade Teste"
Nome fantasia: "Teste Teste "

CONSIDERAÇÕES FINAIS

Para executar esse trabalho foi possível superar muitos desafios passando pelos principais tópicos:

- Configuração do Java 8 e resolução de inúmeras incompatibilidades;
- Instalação e configuração do servidor WildFly compatível com o java;
- Estruturação dos diretórios com a organização correta de pastas e arquivos;
- Inserção de bibliotecas atualizadas manipulação do código;
- Separação das responsabilidades entendendo os conceitos Model, View Controler.
- Entedimento do arquivo XML.

REFERÊNCIAS

- 1) ARAGÃO, Helder Guimarães. *Java e Programação Orientada a Objetos: uma abordagem didática*. 1. ed. Salvador: [s.n.], 2012.
- 2) DEV.MEDIA. Guia completo de JSF (JavaServer Faces): aprenda JSF **agora!** Disponível em: <https://www.devmedia.com.br/guia/jsf-javaserver>. Acesso em: **[março 2026]**
- 3) CONFIGURANDO O SERVIDOR WILDFLY COM A IDE ECLIPSE.
Disponível em: <https://www.youtube.com/watch?v=pKqQXSUypow>. Acesso em: 13 fev. 2026.